

UNIFICACION MEMORIA + RAZONAMIENTO (ASISTENTE C)

Sí. Lo que tienes ahora es una muy buena base para evolucionar hacia un “sistema cognitivo persistente” en lugar de solo prompts estáticos.

La clave es separar 4 capas:

1. Configuración del agente
2. Memoria persistente
3. Razonamiento/orquestación
4. Aprendizaje y consolidación

Tu estructura actual ya cubre parcialmente la capa 1. Ahora puedes extenderla para incorporar conceptos inspirados en Engram sin perder simplicidad.

Te propongo una arquitectura híbrida:

- Tu sistema actual → “Agent Runtime”
- Ideas de Engram → “Memory Engine”

Eso te permite:

- acumular experiencia entre proyectos
- reutilizar conocimiento
- mejorar retrieval
- tener memoria episódica real
- generar aprendizaje automático post-ejecución

1. NUEVA ESTRUCTURA RECOMENDADA

txt

```
mi-proyecto/
├── .ai/
│   ├── system.md
│   ├── rules.md
│   ├── context.md
│   └── agents.md
├── memory/
│   └── schema.md
```

```
|
|
|   ├── policies.md
|   ├── retrieval.md
|   ├── consolidation.md
|   └── reflections.md
|
|   ├── reasoning/
|   │   ├── cot.md
|   │   ├── planning.md
|   │   ├── factcheck.md
|   │   └── tool_usage.md
|   └── knowledge/
|       ├── architecture.md
|       ├── decisions.md
|       ├── patterns.md
|       └── mistakes.md
|
|   ├── memory/
|   │   ├── chroma/
|   │   ├── episodic/
|   │   ├── semantic/
|   │   ├── summaries/
|   │   └── indexes/
|   └── logs/
|       ├── traces/
|       ├── tool_calls/
|       └── reasoning/
|
|   └── src/
|
|   └── opencode.json
```

2. DIFERENCIA CLAVE: MEMORIA ≠ CONTEXTO

Actualmente:

- `.ai/context.md` = contexto estático

Lo nuevo:

- `memory/` = conocimiento dinámico acumulado

Ejemplo:

Contexto:

Markdown

Memoria:

JSON

```
{
  "type": "episode",
  "text": "Migración Prisma falló por lock en Railway",
  "importance": 0.8,
  "solution": "Agregar shadowDatabaseUrl"
}
```

Eso transforma el sistema en uno que:

- recuerda errores
- recuerda decisiones
- aprende patrones
- evita repetir fallos

3. IMPLEMENTAR STM + LTM + EPISÓDICA

Te recomiendo:

STM (Short-Term Memory)

Contexto inmediato de la sesión.

Guardar:

- conversación actual
- plan activo

- tools recientes
- archivos abiertos

Ubicación:

txt

logs/reasoning/current.json

TTL:

- minutos o sesión actual

LTM (Long-Term Memory)

Conocimiento consolidado.

Guardar:

- arquitectura
- patrones útiles
- decisiones recurrentes
- soluciones verificadas

Backend:

- ChromaDB
- SQLite + embeddings
- LanceDB

Memoria Episódica

Experiencias concretas.

Guardar:

- errores
- debugging
- prompts exitosos
- decisiones humanas
- deploys fallidos

Ejemplo:

JSON

```
{
  "type": "episode",
  "task": "Deploy Vercel",
  "problem": "Edge runtime rompía Prisma",
  "solution": "Usar Node runtime",
  "outcome": "success"
}
```

Esta es la memoria más importante para “mejorar con experiencia”.

4. ESQUEMA TIPADO DE MEMORIA

Lo que muestras de Pydantic es excelente.

Te recomiendo formalizarlo en:

txt

`.ai/memory/schema.md`

y además implementarlo en código:

TypeScript

```
export interface MemoryRecord {  
  id: string  
  type: "fact" | "episode" | "decision" | "pattern"  
  text: string  
  summary?: string  
  tags: string[]  
  importance: number  
  confidence: number  
  timestamp: string  
  project?: string  
  source?: string  
  metadata?: Record<string, any>  
}
```

Esto es CRÍTICO porque:

- evita memoria caótica
- permite filtros
- mejora retrieval
- habilita ranking

5. CAPA DE REFLEXIÓN (MUY IMPORTANTE)

Aquí está la diferencia entre “guardar logs” y “aprender”.

Después de cada tarea:

El agente ejecuta:

txt

reflection()

Y genera:

- qué salió bien
- qué salió mal
- qué patrón apareció
- qué debería recordarse

- qué puede descartarse

Archivo:

txt

.ai/memory/reflections.md

Ejemplo:

Markdown

Patrón detectado:
En proyectos Next.js + Prisma:
- usar runtime=nodejs
- evitar edge runtime

Confianza: alta

Esto convierte experiencia → conocimiento reusable.

6. CONSOLIDACIÓN AUTOMÁTICA

Inspirado en Engram.

Necesitas un proceso periódico:

txt

memory_consolidation()

Funciones:

- resumir memorias repetidas
- eliminar duplicados
- aumentar importance
- archivar memorias viejas
- generar patrones

Ejemplo:

5 errores similares →
1 patrón consolidado

Antes:

```
txt
```

```
Error Prisma x5
```

Después:

```
txt
```

Pattern:

```
Prisma falla en Edge runtimes
```

7. RETRIEVAL INTELIGENTE

Ahora mismo probablemente haces:

- similarity search simple

Puedes mejorar con:

a) Metadata filtering

JSON

```
{  
  "project": "nextjs",  
  "type": "episode"
```

```
}
```

b) Importance ranking

Score:

```
txt
```

```
similarity * importance * recency
```

c) Query rewriting

Pregunta:

```
txt
```

```
"deploy error"
```

Reescribir:

```
txt
```

```
"deployment failures vercel railway docker prisma"
```

d) Hybrid retrieval

- embeddings
- BM25 keyword search
- tags
- metadata

8. FACT-CHECKING + CONFIANZA

Tu idea aquí ya supera a Engram.

Añade:

JSON

```
{
  "confidence": 0.91,
  "verified": true,
  "sources": [...]
}
```

Y bloquea respuestas con:

txt

confidence < 0.4

Eso reduce alucinaciones muchísimo.

9. GRAFO DE CONOCIMIENTO

Nivel avanzado.

Relacionar memorias:

txt

Prisma

├─ Railway
├─ Edge Runtime

├─ PostgreSQL

Esto permite reasoning contextual real.

Puedes empezar simple:

JSON

```
{  
  "related_to": ["prisma", "vercel", "deploy"]  
}
```

10. EL CAMBIO MÁS IMPORTANTE

Ahora mismo:

- tu `.ai/` define comportamiento

Con memoria:

- el sistema desarrolla experiencia

Eso transforma el agente de:

txt

Prompt-engineered

a:

txt

Experience-driven

11. ROADMAP RECOMENDADO

FASE 1

- schemas tipados

- memoria episódica
- embeddings persistentes

FASE 2

- retrieval híbrido
- ranking
- reflections automáticas

FASE 3

- consolidación
- deduplicación
- importance decay

FASE 4

- knowledge graph
- multi-agent memory sharing
- self-improving prompts

12. RECOMENDACIÓN FINAL

Tu arquitectura ideal sería:

txt

```
[ opencode agent runtime ]
    ↓
[ reasoning/orchestrator ]
    ↓
[ memory engine ]
    ├── STM
    ├── episodic
    ├── semantic/LTM
    └── reflections

    └── consolidation
```

No intentaría copiar Engram completo.

Tomaría:

- schemas
- lifecycle
- retrieval
- consolidation

Y mantendría:

- tu orchestrator
- CoT
- fact-checking
- tools
- opencode integration

Porque ahí está tu ventaja real.